数值分析考题生成、求解与优化全流程报告

2026 年 1 月 11 日

1 一、整体流程回顾

本次任务严格遵循“考题生成—交叉求解—答案校验—题目优化”的闭环流程，充分利用两款大模型的优势完成数值分析核心考点的考题设计与验证，具体流程如下：

1. **考题生成**：通过 Gemini 生成覆盖矩阵特征值、多项式插值、函数逼近、数值积分与微分、常微分方程数值解 5 大核心考点的模拟试题，题目兼顾理论证明与计算分析，符合数值分析课程考试的核心要求。

2. **交叉求解**：由 DeepSeek 对 Gemini 生成的考题进行详细解答，输出完整解题步骤与结果，形成初步解答方案。

3. **答案校验**：Gemini 对 DeepSeek 的解答进行逐题校验，识别出计算错误（如第三题最佳平方逼近系数求解错误）与表述不严谨问题，并给出正确答案与修正建议。

4. **二次验证与优化**：DeepSeek 认可 Gemini 的修正建议，通过 Python 代码验证正确结果，同时双方围绕题目难度、考察深度、实用性等维度提出优化方案，最终形成高质量考题与标准解答。

2 二、大模型表现点评

2.1 （一）Gemini（考题生成与校验方）

2.1.1 优势

1. **考题设计质量高**：题目覆盖全面，精准命中数值分析核心考点；题型创新度高，如 Hermite 插值设计“一点含函数值与导数值、一点仅含函数值”的非对称场景，避免学生死记公式，侧重考察算法本质；难度层次分明，理论证明与计算分析结合，符合高水平考试风格。

2. **校验能力精准**：能够快速识别 DeepSeek 解答中的低级计算错误（如第三题法方程组求解错误）与表述瑕疵（如插值余项中 ξ 的处理不严谨），给出的修正意见具体且有依据，同时提供详细的标准解答，逻辑严谨。

3. **优化建议专业**：针对考题存在的计算陷阱、考察深度不足等问题，提出“强制使用正交基减少计算错误”“明确误差界计算”“增加构造性难度”等优化方案，直击题目设计核心痛点，提升考题的区分度与考察价值。

2.1.2 不足

1. 部分题目表述可进一步完善，如 Hermite 插值中“不知道导数值”的表述易引发歧义，需更明确说明是“仅已知函数值，无导数信息”。

2. 未在初始题目中强化知识点间的关联，如插值误差与数值积分外推的结合，缺乏综合应用类题目。

2.2 （二）DeepSeek（考题求解与反馈方）

2.2.1 优势

1. **理论推导能力强**：在瑞利商驻点证明、梯形公式 A-稳定性证明等理论题中，推导步骤严谨，逻辑链条清晰，准确应用数值分析核心原理，展现了扎实的理论基础。

2. **算法应用熟练**：能够正确构造 Hermite 插值均差表、高斯积分节点与权系数、Richardson 外推公式等，对数值算法的核心步骤掌握熟练，无概念性错误。

3. **纠错意识积极**：在 Gemini 指出计算错误后，能够快速承认问题，通过手动重算与 Python 代码验证两种方式确认正确结果，并分析错误原因，体现了良好的纠错能力。

2.2.2 不足

1. 存在低级计算失误，如第三题法方程组求解时，因手工计算疏忽导致 a_0 取值错误（正确值为 -0.3，错误值为 -0.6），反映出手工计算的不稳定性。

2. 部分解答表述不严谨，如插值余项中未明确 ξ 的取值范围，仅给出含混的表达式，缺乏数学严谨性。

3. 对题目设计的优化建议针对性不足，更多聚焦于表面表述调整，未深入挖掘考题对算法本质的考察潜力。

3 三、考题优化前后对比

考题模块	优化前问题	优化方案	优化后效果
函数逼近	允许直接解法方程组，易因手工计算出错；误差分析仅要求列出公式，考察深度不足	强制使用正交基进行最佳平方逼近；要求利用正交多项式性质简化误差计算	减少计算陷阱，突出“正交投影”核心思想；提升误差分析的考察深度，答案唯一性更强
多项式插值	误差表达式要求模糊，学生易给出含混答案	将“写出误差表达式”改为“求出区间 $[0,1]$ 上的最大截断误差界”	考察学生对导数极值的分析能力，强化数学严谨性
常微分方程数值解	零稳定性仅要求判断给定参数，难度较低；缺乏显式/隐式概念强化	改为“要求方法零稳定时参数需满足的关系，并给出高阶参数”；增加“显式方法的条件”小问	提升构造性难度，强化对根条件的理解；深化显式/隐式方法的概念辨析
整体结构	缺乏知识点融合，无综合应用题	增加综合应用题，涵盖有限差分法、圆盘定理、反幂法、步长影响分析	强化知识点间的关联，考察学生综合应用能力

表 1: 考题优化前后对比表

4 四、拓展问题解答

4.1 （一）选题：第三题“最佳平方逼近”

4.1.1 1. 方法原理

本题中最佳平方逼近的求解采用两种方法：

- 初始方法：直接基于幂基 $\{1, x\}$ 构建法方程组 $\boldsymbol{Aa} = \boldsymbol{b}$，通过求解线性方程组得到逼近多项式系数；
- 优化方法：先通过 Gram-Schmidt 正交化将幂基转化为正交基 $\{\phi_0(x), \phi_1(x)\}$，再利用傅里叶系数公式 $c_k = \frac{(f,\phi_k)}{(\phi_k,\phi_k)}$ 求解系数。

4.1.2 2. 时间复杂度分析

直接解法方程组方法 ：

1. 计算内积构建法方程组：对于 n 维子空间（本题 $n=2$），需计算 $(n+1)(n+2)/2$ 个内积，每个内积为定积分计算，数值积分时时间复杂度为 $O(M)$（M 为积分节点数，通常取固定值，可视为常数），因此构建矩阵的时间复杂度为 $O(1)$（子空间维度固定时）；
2. 求解线性方程组：对于 n 阶线性方程组，采用高斯消元法的时间复杂度为 $O(n^3)$。本题 $n=2$，时间复杂度为 $O(1)$；
3. 总体时间复杂度：当子空间维度 n 固定时，整体时间复杂度为 $O(1)$；若子空间维度 n 增加，时间复杂度为 $O(n^3)$。

正交基方法 ：

1. Gram-Schmidt 正交化：对于 n 个基向量，正交化过程中需计算 $O(n^2)$ 个内积，时间复杂度为 $O(n^2)$（内积计算视为常数）；
2. 计算傅里叶系数：共 n 个系数，每个系数需计算 2 个内积，时间复杂度为 $O(n)$；
3. 总体时间复杂度：$O(n^2)$，远低于直接解法方程组的 $O(n^3)$。

4.1.3 3. 数据规模扩大 1000 倍后的适用性分析

此处“数据规模扩大 1000 倍”可理解为**最佳平方逼近的子空间维度 n 扩大 1000 倍**（即从 $n=2$ 变为 $n=2000$）：

- **直接解法方程组方法**：时间复杂度 $O(n^3)$，当 $n=2000$ 时，计算量为 $(2000)^3 = 8\times10^9$ 次运算，计算成本极高，且法方程组为希尔伯特矩阵，数值稳定性差，易出现溢出现象，因此**完全不适用**；
- **正交基方法**：时间复杂度 $O(n^2)$，当 $n=2000$ 时，计算量为 $(2000)^2 = 4\times10^6$ 次运算，虽计算量增加，但仍在可接受范围内，但当 n 继续增大（如 $n=10^5$）时，$O(n^2)$ 的时间复杂度也将面临计算瓶颈，因此**在中等规模下适用，大规模下不适用**。

4.1.4 4. 工业界替代算法

当数据规模（子空间维度 n）极大时，工业界通常采用以下替代算法：

1. **基于快速傅里叶变换（FFT）的逼近算法**：

- 适用场景：当权函数为常数（如 $\rho(x)=1$）且正交基为三角函数系（傅里叶基）时，可通过 FFT 快速计算傅里叶系数，时间复杂度从 $O(n^2)$ 降至 $O(n\log n)$，大幅提升计算效率；
- 核心优势：利用三角函数系的正交性与周期性，通过 FFT 减少内积计算的冗余操作，适用于大规模信号处理、图像处理等场景。

2. **稀疏逼近算法（如正交匹配追踪 OMP）**：

- 适用场景：当逼近函数可由低维稀疏正交基表示时，无需构建完整的高维正交基，通过贪心算法逐步选择最优基向量，仅需 $O(kn)$ 次运算（k 为稀疏度，$k \ll n$）；
- 核心优势：避免高维空间的正交化与内积计算，降低时间复杂度与空间复杂度，适用于机器学习、数据分析等大规模稀疏数据场景。

3. **随机投影算法**：

- 适用场景：高维空间中的最佳平方逼近问题，通过随机投影将高维数据映射到低维空间，再在低维空间中进行逼近计算，时间复杂度为 $O(nm)$（m 为低维空间维度，$m \ll n$）；
- 核心优势：基于约翰逊-林登施特劳斯引理，保证投影后数据的距离近似不变，在大规模数据挖掘、推荐系统等场景中广泛应用。

5 五、总结

本次通过双大模型协作完成的数值分析考题生成与求解任务，充分发挥了不同模型在题目设计、理论推导、计算验证等方面的优势，最终形成了高质量的考题与标准解答。过程中暴露的手工计算错误、题目表述歧义等问题，通过交叉校验与代码验证得到有效解决，为数值分析课程的考试命题与学习验证提供了高效方案。同时，针对拓展问题的分析表明，数值算法的适用性不仅取决于精度，还需兼顾计算成本，工业界在处理大规模问题时，会根据问题特性选择更高效的稀疏算法、快速变换算法等替代方案，平衡精度与效率的关系。

6 附录：DeepSeek 代码验证（第三题）

```python
import numpy as np

# 定义权函数rho(x)=x，计算加权内积(g,h)_rho = ∫_0^1 g(x)h(x)x dx
def inner_product(g, h, a=0, b=1):
    """计算加权内积"""
    # 使用高斯积分保证精度，映射区间[-1,1]到[0,1]
    x, w = np.polynomial.legendre.leggauss(100)
    t = 0.5 * (x + 1)  # 映射到[0,1]
    dt_dx = 0.5        # 雅可比行列式
    return np.sum(g(t) * h(t) * t * w * dt_dx)

# 定义基函数和目标函数
def phi0(x):
    return np.ones_like(x)

def phi1(x):
    return x

def f(x):
    return x ** 2

# 计算法方程组系数矩阵A和右端向量b
A = np.array([
    [inner_product(phi0, phi0), inner_product(phi0, phi1)],
    [inner_product(phi1, phi0), inner_product(phi1, phi1)]
])
b = np.array([
    inner_product(f, phi0),
    inner_product(f, phi1)
])

# 求解线性方程组
coeff_numeric = np.linalg.solve(A, b)
print("数值解：a0={:.6f}, a1={:.6f}".format(coeff_numeric[0], coeff_numeric
    [1]))

# 解析解验证
A_analytic = np.array([[1/2, 1/3], [1/3, 1/4]])
b_analytic = np.array([1/4, 1/5])
coeff_analytic = np.linalg.solve(A_analytic, b_analytic)
print("解析解：a0={:.6f}, a1={:.6f}".format(coeff_analytic[0],
    coeff_analytic[1]))

```

```
# 计算相对误差
relative_error_a0 = abs((coeff_numeric[0] - coeff_analytic[0]) /
    coeff_analytic[0]) * 100
relative_error_a1 = abs((coeff_numeric[1] - coeff_analytic[1]) /
    coeff_analytic[1]) * 100
print("相对误差: a0={:.2f}%, a1={:.2f}%".format(relative_error_a0,
    relative_error_a1))
```

Listing 1: 最佳平方逼近系数计算验证

```
数值解: a0=-0.300000, a1=1.200000
解析解: a0=-0.300000, a1=1.200000
相对误差: a0=0.00%, a1=0.00%
```

Listing 2: 代码运行结果